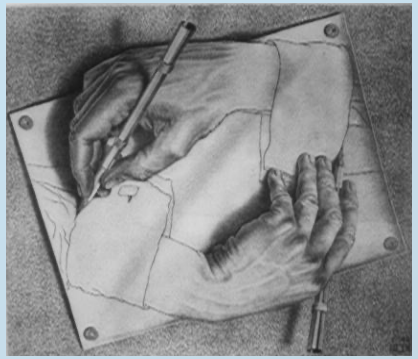
Αναδρομή
(Recursion)

Η Δίδυμη Αδελφή της Επανάληψης



Reproductive Parts
M. C. Escher, 1948

Αναδρομή

- Αναδρομή είναι όταν μία συνάρτηση **καλεί τον εαυτό της** άμεσα ή έμμεσα
- Γιατί είναι σημαντικό/χρήσιμο να μάθουμε αναδρομή;
  - Νέος τρόπος σκέψης
  - Ισχυρό παράδειγμα προγραμματισμού
- Πολλοί υπολογισμοί είναι από τη φύση τους αυτο-αναφορικοί
  - Mergesort, FFT (Fast Fourier Transform), gcd (μέγιστος κοινός διαιρέτης)
  - Συνδεδεμένες δομές δεδομένων
  - Ένας φάκελος που περιέχει αρχεία και άλλους φακέλους
- Στενά συνδεμένη με τη μαθηματική επαγωγή (mathematical induction)

Ανάλυση Περιπτώσεων

- Όταν εφαρμόζουμε αναδρομή αναλύουμε τις σχετικές περιπτώσεις σε
  - Βασικές ή τερματικές περιπτώσεις (base case)
  - Γενικές ή μη-τερματικές (αναδρομικές) περιπτώσεις
    - Οι αναδρομικές κλήσεις φέρνουν μια μη-τερματική περίπτωση ένα βήμα πιο κοντά σε κάποια από τις τερματικές περιπτώσεις
- Η ανάλυση περιπτώσεων συνδέεται με επιλογή

- Επαναλήψεις μπορεί να αναδιατυπωθούν ως αναδρομές και το αντίθετο:
  - Ποιος είναι ο πιο φυσικός τρόπος διατύπωσης;
  - Προκύπτουν και ζητήματα υπολογιστικού κόστους

Μέγιστος Κοινός Διαιρέτης

Μέγιστος κοινός διαιρέτης των αριθμών α και β είναι ο μεγαλύτερος κοινός διαιρέτης των α και β. Ο μέγιστος κοινός διαιρέτης των α, β συμβολίζεται με ΜΚΔ(α,β)

$$4032 = 2^6 \times 3^2 \times 7^1$$
$$1272 = 2^3 \times 3^1 \times 53^1$$
$$\text{ΜΚΔ} = 2^3 \times 3^1 = 24$$

π.χ. ΜΚΔ(4032, 1272) = 24.

Εφαρμογές:

- Απλοποίηση κλασμάτων: 1272/4032 = 53/168
- RSA κρυπτογραφικό σύστημα

Μέγιστος Κοινός Διαιρέτης

- ΜΚΔ: Να βρεθεί ο μεγαλύτερος κοινός διαιρέτης των p και q.
- Ευκλείδειος αλγόριθμος [Ευκλείδης 300 πΧ]

$$\gcd(p, q) = \begin{cases} p & \text{if } q = 0 \\ \gcd(q,\ p \ \%\ q) & \text{otherwise} \end{cases}$$

← Βασική περίπτωση

← Βήμα αναδρομής, συγκλίνει στη βασική περίπτωση



```
gcd(4032, 1272)  = gcd(1272, 216)
                 = gcd(216, 192)
                 = gcd(192, 24)
                 = gcd(24, 0)
                 = 24.
```

```
4032 = 3 × 1272 + 216
```

Το πιο γνωστό έργο του είναι τα ***Στοιχεία***, που αποτελείται από 13 βιβλία. Το έργο του Ευκλείδη ήταν τόσο σημαντικό ώστε η γεωμετρία που περιέγραψε στα *Στοιχεία* του (η βάση της οποίας είναι: έστω μία ευθεία ε και ένα σημείο Α όχι πάνω σε αυτήν την ευθεία, τότε υπάρχει μόνο μία ευθεία, παράλληλη της ε, που διέρχεται από το Α) ονομάστηκε Ευκλείδεια, ενώ τα *Στοιχεία* σήμερα θεωρούνται ένα από τα σημαντικότερα μαθηματικά έργα όλων των εποχών. Όταν ο Πτολεμαίος Α΄ του ζήτησε έναν πιο εύκολο τρόπο από τα *Στοιχεία* του για να μάθει Γεωμετρία η απάντηση του μεγάλου μαθηματικού ήταν: ***«Δεν υπάρχει βασιλική οδός για τη Γεωμετρία»***.

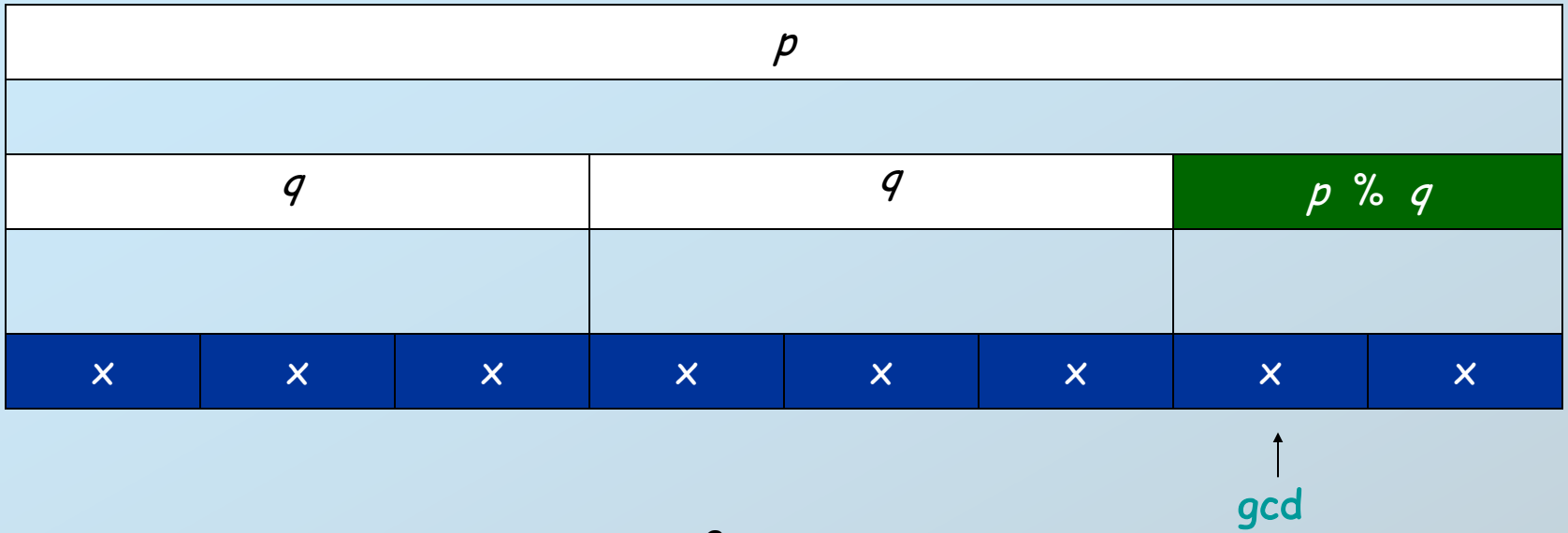
Μέγιστος Κοινός Διαιρέτης

ΜΚΔ: Να βρεθεί ο μεγαλύτερος κοινός διαιρέτης των p και q.

$$\gcd(p, q) = \begin{cases} p & \text{if } q = 0 \\ \gcd(q,\ p \ \%\ q) & \text{otherwise} \end{cases}$$

← Βασική περίπτωση

← Βήμα αναδρομής, συγκλίνει στη βασική περίπτωση



p = 8x
q = 3x
gcd(p, q) = x

Μέγιστος Κοινός Διαιρέτης

ΜΚΔ: Να βρεθεί ο μεγαλύτερος κοινός διαιρέτης των p και q.

$$\gcd(p, q) = \begin{cases} p & \text{if } q = 0 \\ \gcd(q,\ p \ \%\ q) & \text{otherwise} \end{cases}$$

← Βασική περίπτωση

← Βήμα αναδρομής, συγκλίνει στη βασική περίπτωση

```
public static int gcd(int p, int q) {
   if (q == 0) return p;
   else return gcd(q, p % q);
}
```

← Βασική περίπτωση

← Βήμα αναδρομής

Παραγοντικό

Συνάρτηση για τον υπολογισμό του παραγοντικού (factorial) ενός φυσικού αριθμού:

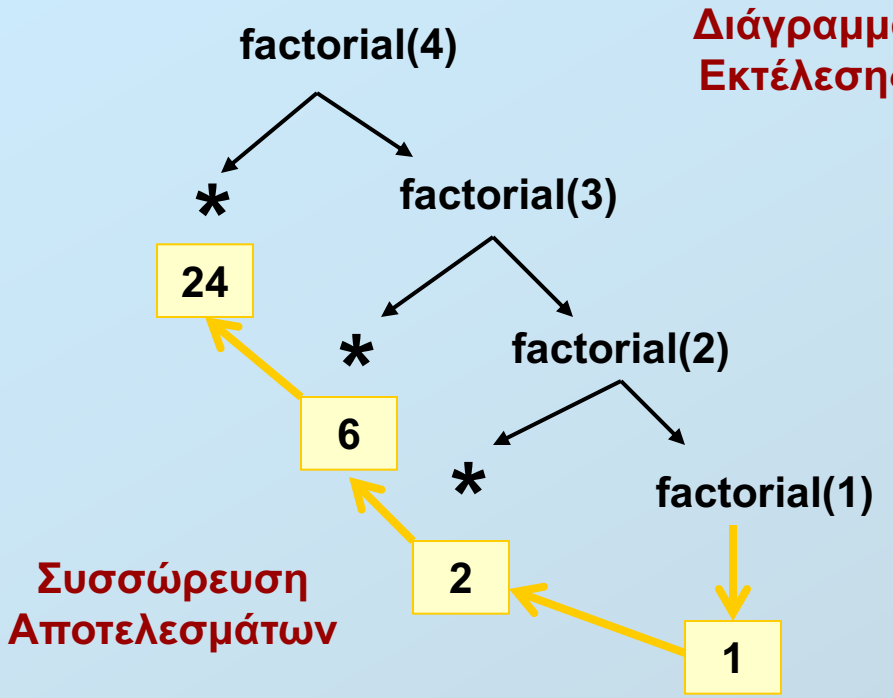
$$0! = 1$$
$$1! = 1$$
$$2! = 1 \times 2$$
$$3! = 1 \times 2 \times 3 = 2! \times 3$$
$$\ldots\ldots\ldots$$
$$n! = 1 \times 2 \times \ldots \times (n-1) \times n$$
$$= (n-1)! \times n$$

```
/* Iterative definition */
public static int factorial (int x){
    /* assumes x ≥ 0 */
    int res = 1;
    while (x > 1) {res = res * x;  x = x - 1;}
    return res;
 }
```

```
/* (Stack) Recursive definition */
public static int factorial (int x){
   if (x == 0 || x == 1) return 1;
   else return x * factorial(x - 1); }
```

```
factorial(4)
⇒ 4 * factorial(3)
⇒ 4 * 3 * factorial(2)
⇒ 4 * 3 * 2 * factorial(1)
⇒ 4 * 3 * 2 * 1
⇒ 24
```

Αναδρομή στοίβας


Διάγραμμα
Εκτέλεσης
factorial(4)
*
factorial(3)
24
*
factorial(2)
6
*
factorial(1)
Συσσώρευση
Αποτελεσμάτων
2
1

```
/* (Tail) Recursive definition */
public static int factorial (int x){
    return factorial(x,1);
}

// Function overloading
private static int factorial (int x, int res){
    if (x == 0 || x == 1) return res;
    else return factorial(x-1, res*x); }
```

```
factorial(4)
⇒ factorial(4,1)
⇒ factorial(3,4)
⇒  factorial(2,12)
⇒  factorial(1,24)
⇒ 24
```



Αναδρομή ουράς

«Ισοδύναμη» με επανάληψη

Επισημάνσεις αναφορικά με την Αναδρομή

1. **Διαχωρισμός ανάμεσα σε**
   - **τερματικές (βασικές) και**
   - **αναδρομικές (γενικές) περιπτώσεις**

   με χρήση εντολής επιλογής

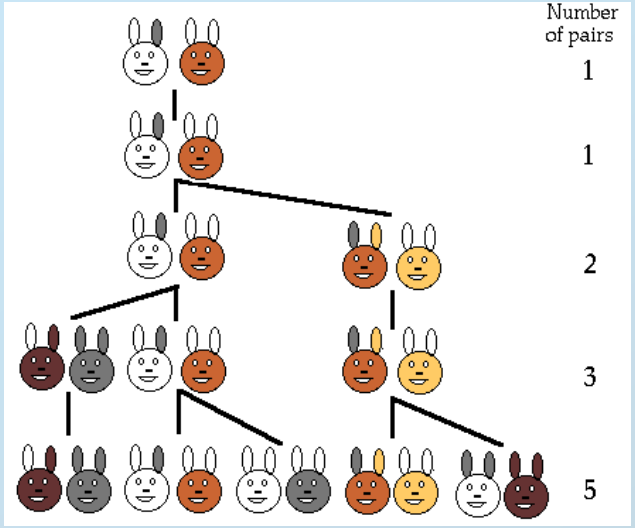
2. **Η αναδρομική κλήση πρέπει να είναι ένα βήμα πιο κοντά σε τερματική περίπτωση, από την κλήση που οδήγησε σε αυτήν**

3. **«Ισοδυναμία» με επανάληψη**

Αριθμοί Fibonacci

Αριθμοί Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, …

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{otherwise} \end{cases}$$



Fibonacci λαγοί



L. P. Fibonacci
(1170 - 1250)

Μία Πιθανή Παγίδα με την Αναδρομή

Fibonacci αριθμοί: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, …

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{otherwise} \end{cases}$$

Είναι φυσική η αναδρομή;

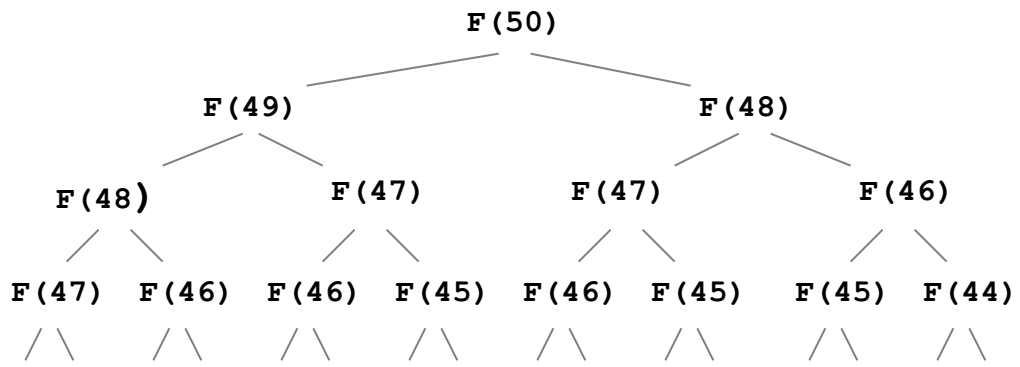
```
public static long F(int n) {
   if (n == 0) return 0;
   else if (n == 1) return 1;
   else return F(n-1) + F(n-2);
}
```

Πρόκληση 1 με την Αναδρομή
(δύσκολη αλλά σημαντική)

Ερώτηση: Είναι αυτός ένας αποτελεσματικός τρόπος για να υπολογιστεί η F(50);

```
public static long F(int n) {
   if (n == 0) return 0;
   else if (n == 1) return 1;
   else return F(n-1) + F(n-2);
}
```

Απάντηση: Όχι, όχι, όχι! Αυτός ο κώδικας είναι **ιδιαίτερα μη αποτελεσματικός**



recursion tree for naïve Fibonacci function

F(50) καλείται μία φορά.

F(49) καλείται μία φορά.

F(48) καλείται 2 φορές.

F(47) καλείται 3 φορές.

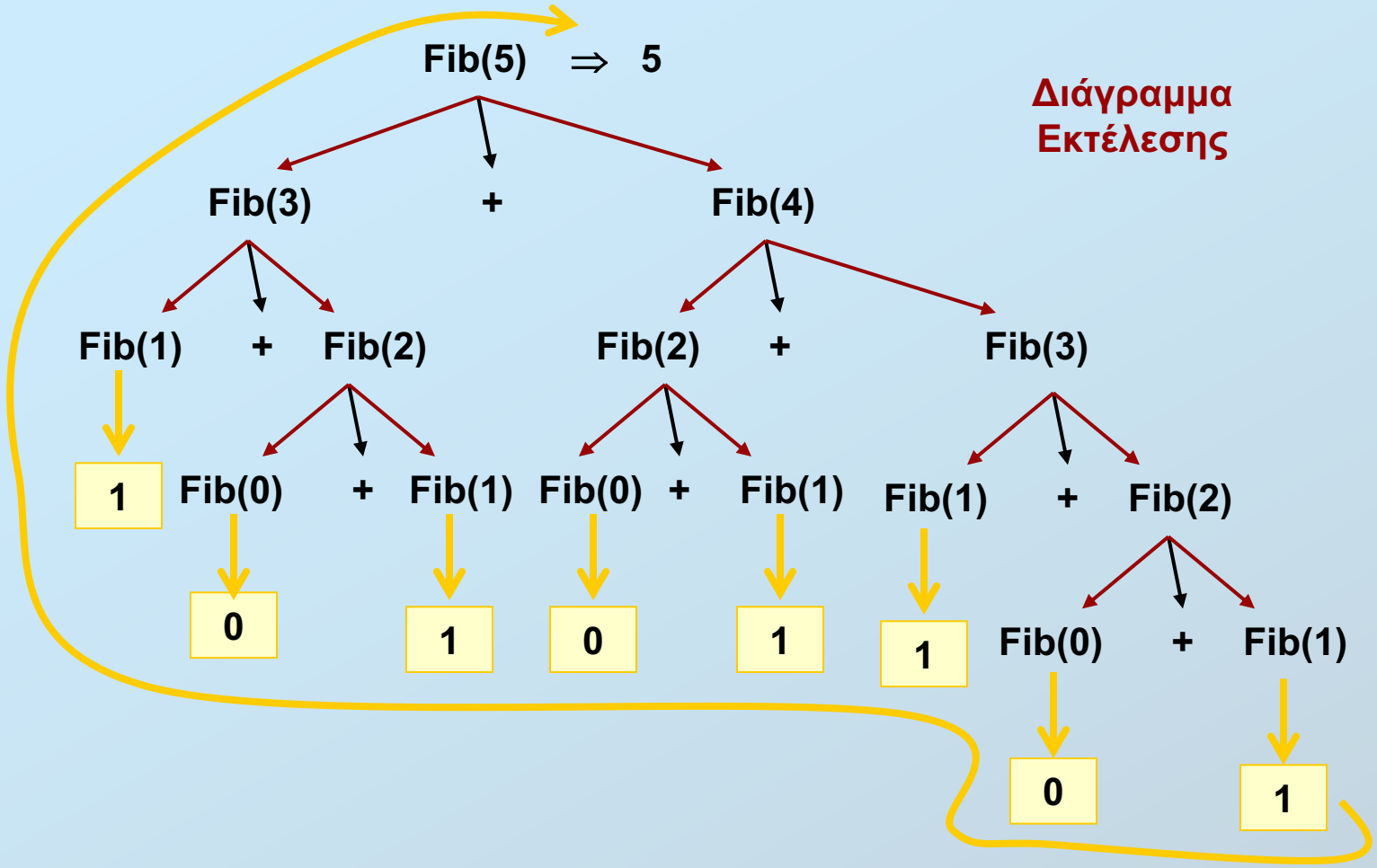
F(46) καλείται 5 φορές.

F(45) καλείται 8 φορές.

...

F(1) καλείται 12,586,269,025 φορές.

F(50)



Όπως ήδη αναφέρθηκε, ο ορισμός με τη διπλή αναδρομή στοίβας δεν είναι αποδοτικός

Πρόκληση 2 με την Αναδρομή

(εύκολη αλλά επίσης σημαντική)

Ερώτηση: Είναι αυτός ένας αποτελεσματικός τρόπος για να υπολογιστεί η F(50);

```
public static long Fib (int n) {
   long[] F = new long[n+1];
   F[0] = 0; F[1] = 1;
   for (int i = 2; i <= n; i++)
      F[i] = F[i-1] + F[i-2];
   return F[n];
}
```

Απάντηση: Ναι. Αυτός ο κώδικας υπολογίζει το Fib(50) με 50 προσθήκες

Δίδαγμα: Μην χρησιμοποιείτε την αναδρομή για να εμπλακείτε σε εκθετική σπατάλη

Πλαίσιο: Αυτό είναι μία ειδική περίπτωση μίας σημαντικής τεχνικής στον προγραμματισμό γνωστής ως δυναμικός προγραμματισμός

Class Fib

- Η κλάση Fib περιλαμβάνει τρεις εναλλακτικούς ορισμούς για τον υπολογισμό όρων της ακολουθίας Fibonacci:
  - **`fibTail`**
    - Αναδρομή ουράς
    - Function overloading (public / private)
  - **`fibStack`**
    - Αναδρομή στοίβας
  - **`fibIter`**
    - Με επανάληψη (while)
- Η main αξιοποιεί την System.nanoTime() και την DrawGraph.plot(..) για να μετρήσει και να παρουσιάσει γραφικά το χρόνο εκτέλεσης των συναρτήσεων **`fibTail`**, **`fibStack`** και **`fibIter`**:
  - **`fibTail`** και **`fibIter`** «ισοδύναμες» υπολογιστικά

```
public class Fib

_____________________________________________________________________

    long fibTail (int x)        returns fib(x)
    long fibStack (int x)       returns fib(x)
    long fibIter (int x)        returns fib(x)
```

```
public class Fib{

 // tail recursive definition
 public static long fibTail (int x){
  return fibTail(0,1,x);
 }

 private static long fibTail (long a, long b, int x){
   if (x==0) return a;
   else return fibTail(b, a+b, x-1);
 }

 // stack recursive definition
 public static long fibStack (int x){
  if (x==0) return 0;
  else if (x==1) return 1;
  else return fibStack(x-1) + fibStack(x-2);
 }

// class definition continues . . .
```

```
// iterative definition
public static long fibIter (int x){
  long a = 0, b = 1, temp;
  while (x != 0){
    temp = a;
    a = b;
    b = b + temp;
    x--;
  }
  return a;
 }

 public static void main(String[] args){
  // T - fibonacci terms from 0 to T
  // Complete definition as illustrated below
  int T = Integer.parseInt(args[0]);
     . . . . . .
 }
}
```

```
// tail recursive definition
public static long fibTail (int x){
  return fibTail(0,1,x);
 }

 private static long fibTail (long a,
                long b, int x){
   if (x==0) return a;
   else return fibTail(b, a+b, x-1);
```

```
fibTail(5) ⇒
fibTail(0,1,5) ⇒
fibTail(1,1,4) ⇒
fibTail(1,2,3) ⇒
fibTail(2,3,2) ⇒
fibTail(3,5,1) ⇒
fibTail(5,8,0) ⇒
5

επιστροφή της τιμής της a
```

```
// iterative definition
public static long fibIter (int x){
  long a = 0, b = 1, temp;
  while (x != 0){
    temp = a;
    a = b;
    b = b + temp;
    x--;
  }
  return a;
 }
```

```
fibIter(5) ⇒  5

x        a        b

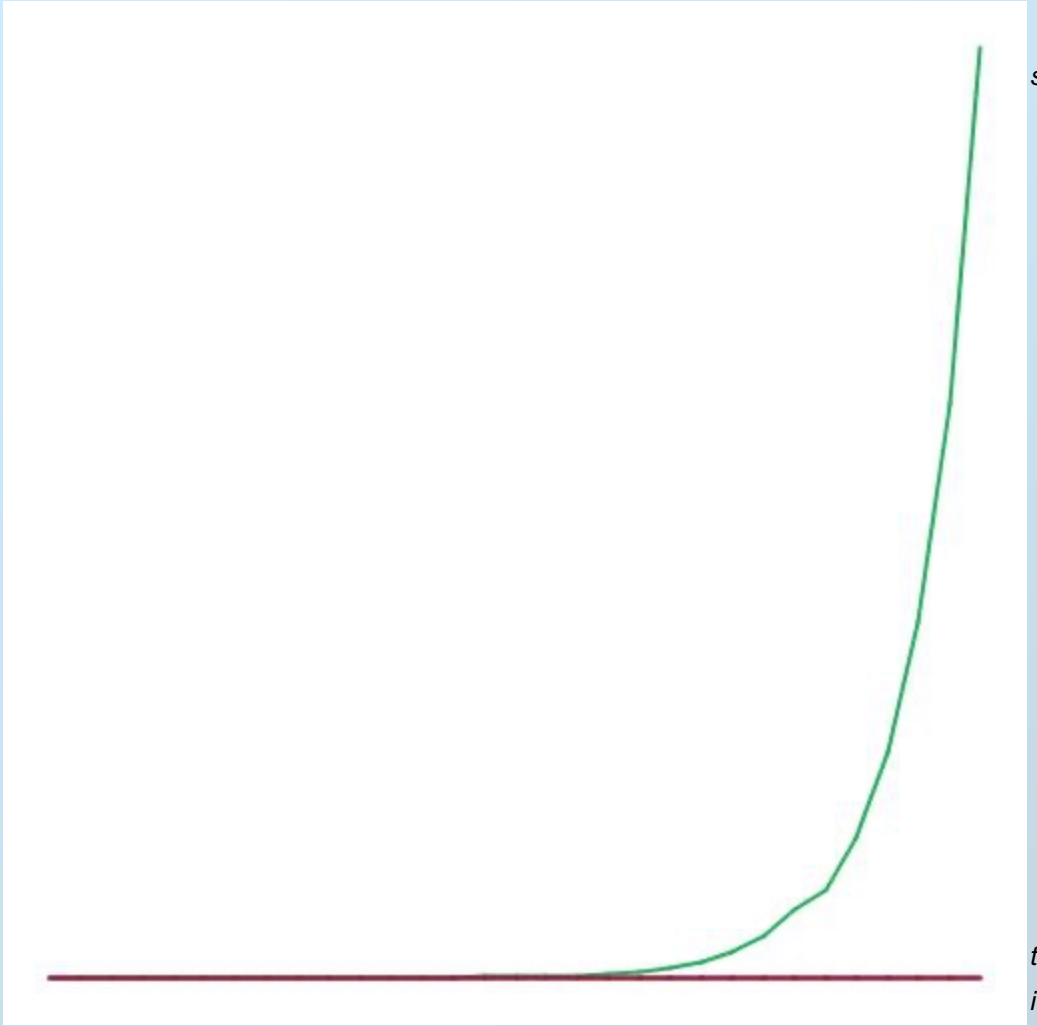
5        0        1
4        1        1
3        1        2
2        2        3
1        3        5
0        5        8

επιστροφή της τιμής της a
```

```
$ java Fib 10
Fib(0)
0          0          0
1823.0     1999.0     1081.0
Fib(1)
1          1          1
233.0      407.0      324.0
Fib(2)
1          1          1
387.0      388.0      255.0
Fib(3)
2          2          2
469.0      396.0      233.0
Fib(4)
3          3          3
695.0      510.0      189.0
Fib(5)
5          5          5
1092.0     594.0      202.0
Fib(6)
8          8          8
1856.0     683.0      218.0
Fib(7)
13         13         13
2942.0     741.0      211.0
Fib(8)
21         21         21
6743.0     811.0      236.0
Fib(9)
34         34         34
13528.0    905.0      279.0
Fib(10)
55         55         55
13287.0    993.0      269.0
```

Execution time
fibStack() fibTail() fibIter()

Fib 0 .. .30 , where each term is computed 50 times (for each function) and execution time recorded and averaged over the 50 times



stack recursive

tail recursive

iterative


String
reverse
String
Hello!
!olleH

```
// Αρχίζουμε με ένα επαναληπτικό ορισμό
public static String reverse (String
s){  String r = "";
for (int i = 0; i < s.length(); i++)  r
= s.substring(i,i+1) + r;
return r;
}
```

reverse(“Hello!”)

Διαδοχικές τιμές της μεταβλητής r	
“”	
H	(“H” + “”)
eH	(“e” + “H”)
leH	(“l” + “eH”)
lleH	(“l” + “leH”)
olleH	(“o” + “lleH”)
!olleH	(“!” + “olleH”)

```
// Προχωρούμε με τον δεύτερο αναδρομικό ορισμό
public static String reverse (String s){
if (s == null || s.length() == 0) return null;
return reverse(s,"");
}

private static String reverse (String s, String r){  if
(s.length() == 1) return s+r;
return reverse(s.substring(1,s.length()), s.substring(0,1)+r);
}
```

reverse("Hello!")

```
reverse("Hello!")
⇒ reverse("Hello!", "")
⇒ reverse("ello!", "H")
⇒ reverse("llo!", "eH")
⇒ reverse("lo!", "leH")
⇒ reverse("o!", "lleH")
⇒ reverse("!", "olleH")
⇒"!olleH"
```



Αναδρομή ουράς

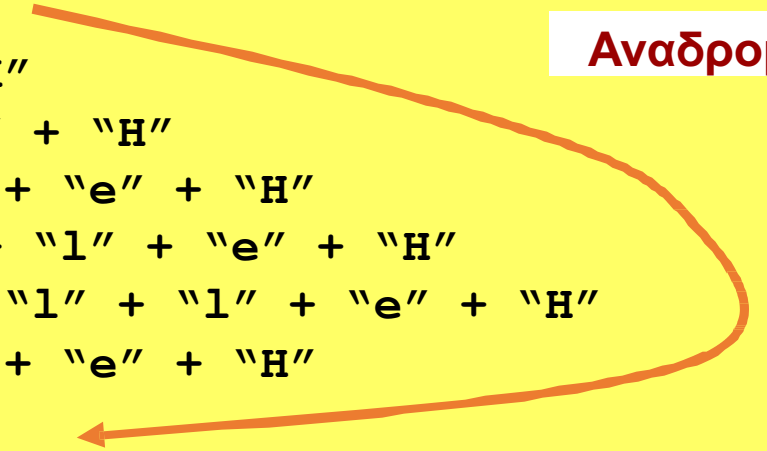
«Ισοδύναμη» με επανάληψη

Function Overloading
Συσσωρευτική Παράμετρος

```
// Συνεχίζουμε με τον πρώτο αναδρομικό ορισμό
public static String reverse (String s)
{
   if (s == null || s.length() == 0) return
   null;  if (s.length() == 1) return s;
   return reverse(s.substring(1,s.length())) +
   s.substring(0,1);
   }
```

reverse("Hello!")

```
reverse("Hello!")
⇒ reverse("ello!") + "H"
⇒ reverse("llo!") + "e" + "H"
⇒ reverse("lo!") + "l" + "e" + "H"
⇒ reverse("o!") + "l" + "l" + "e" + "H"
⇒ reverse("!") + "o" + "l" + "l" + "e" + "H"
⇒"!" + "o" + "l" + "l" + "e" + "H"
⇒"!olleH"
```



Αναδρομή στοίβας

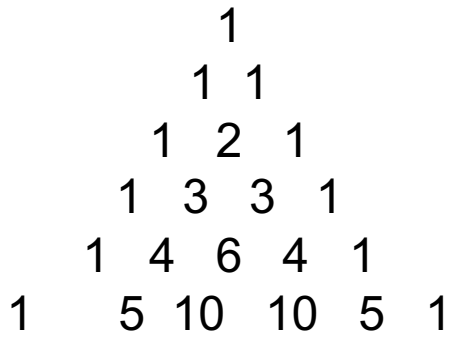
Μνήμη

- Κάθε αναδρομική κλήση δεσμεύει ξεχωριστό χώρο στην μνήμη (έχει το δικό της περιβάλλον εκτέλεσης)
- Αποδεσμεύεται όταν επιστρέψουμε από την κλήση που δέσμευσε τον χώρο
- Πολύ βαθιές αναδρομές κοστίζουν σε χώρο και χρόνο
  - Μπορεί να οδηγήσουν σε Stack overflow εξαίρεση

Παράδειγμα

Το Τρίγωνο του Pascal – Αναδρομική Λύση

Το τρίγωνο του Pascal – Να εκτυπωθούν οι πρώτες N σειρές του τριγώνου




```
%java Recursive_Pascal 10

1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1
1 8 28 56 70 56 28 8 1
1 9 36 84 126 126 84 36 9 1
```

Ορισμός της τιμής του στοιχείου Ε της σειράς R του τριγώνου του Pascal, τ(E,R)

```
%java Recursive_Pascal 10

1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1
1 8 28 56 70 56 28 8 1
1 9 36 84 126 126 84 36 9 1
```

<u>Ανάλυση Περιπτώσεων</u>

1. Αν το Ε είναι το πρώτο στοιχείο της R, η τιμή του είναι το 1:

   τ(E,R)=1 *τερματική περίπτωση*

2. Αν το Ε είναι το τελευταίο στοιχείο της R, η τιμή του είναι και πάλι το 1:

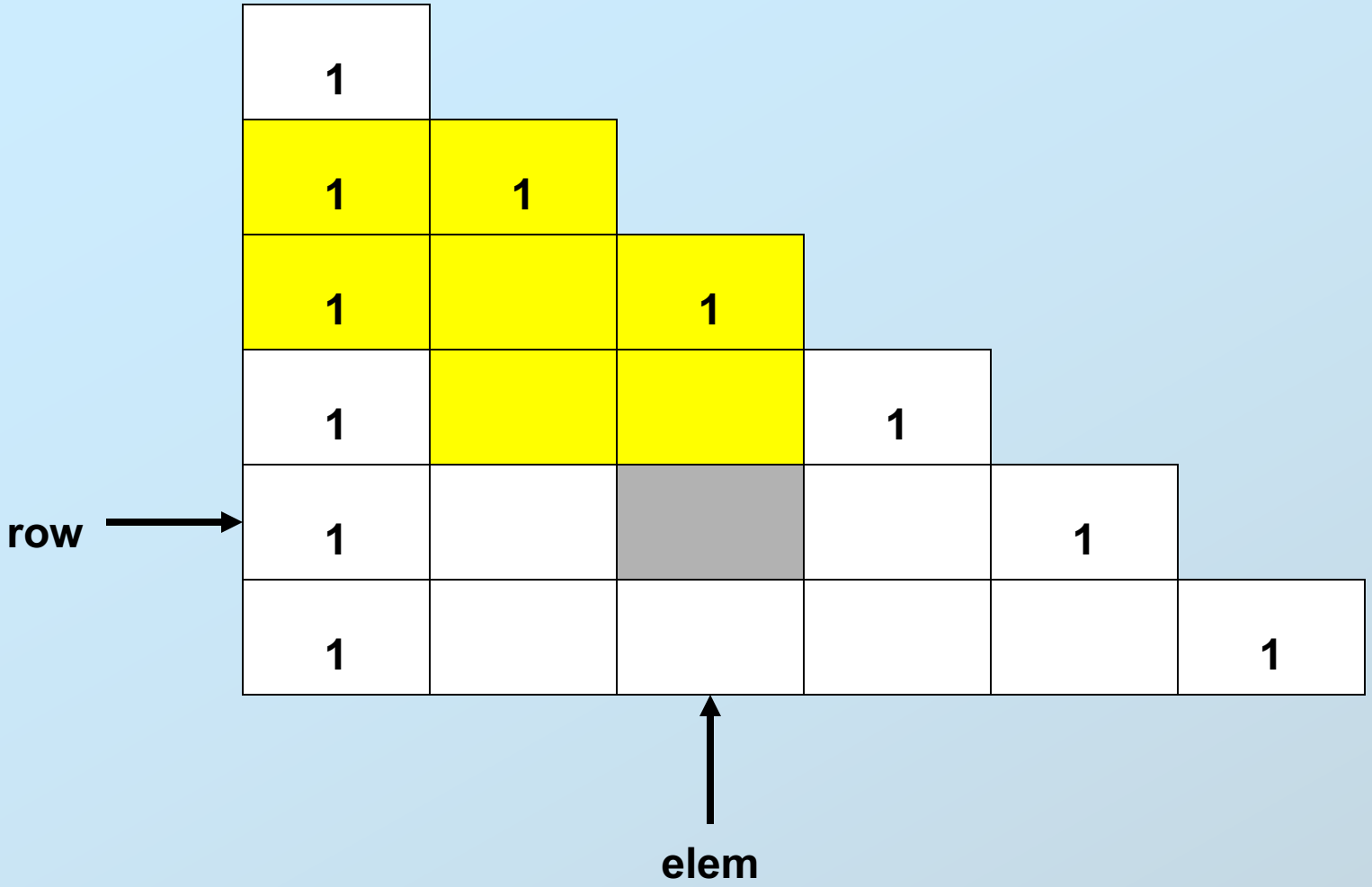
   τ(E,R)=1 *τερματική περίπτωση*

3. Αν το Ε είναι ενδιάμεσο στοιχείο της R:

   τ(E,R) = τ(E,R-1) + τ(E-1,R-1) *μη-τερματική περίπτωση*

Το πλήθος στοιχείων κάθε σειράς ισούται με τον αριθμό της, π.χ. η Ν-οστή σειρά έχει Ν στοιχεία

compute_element(row, elem)



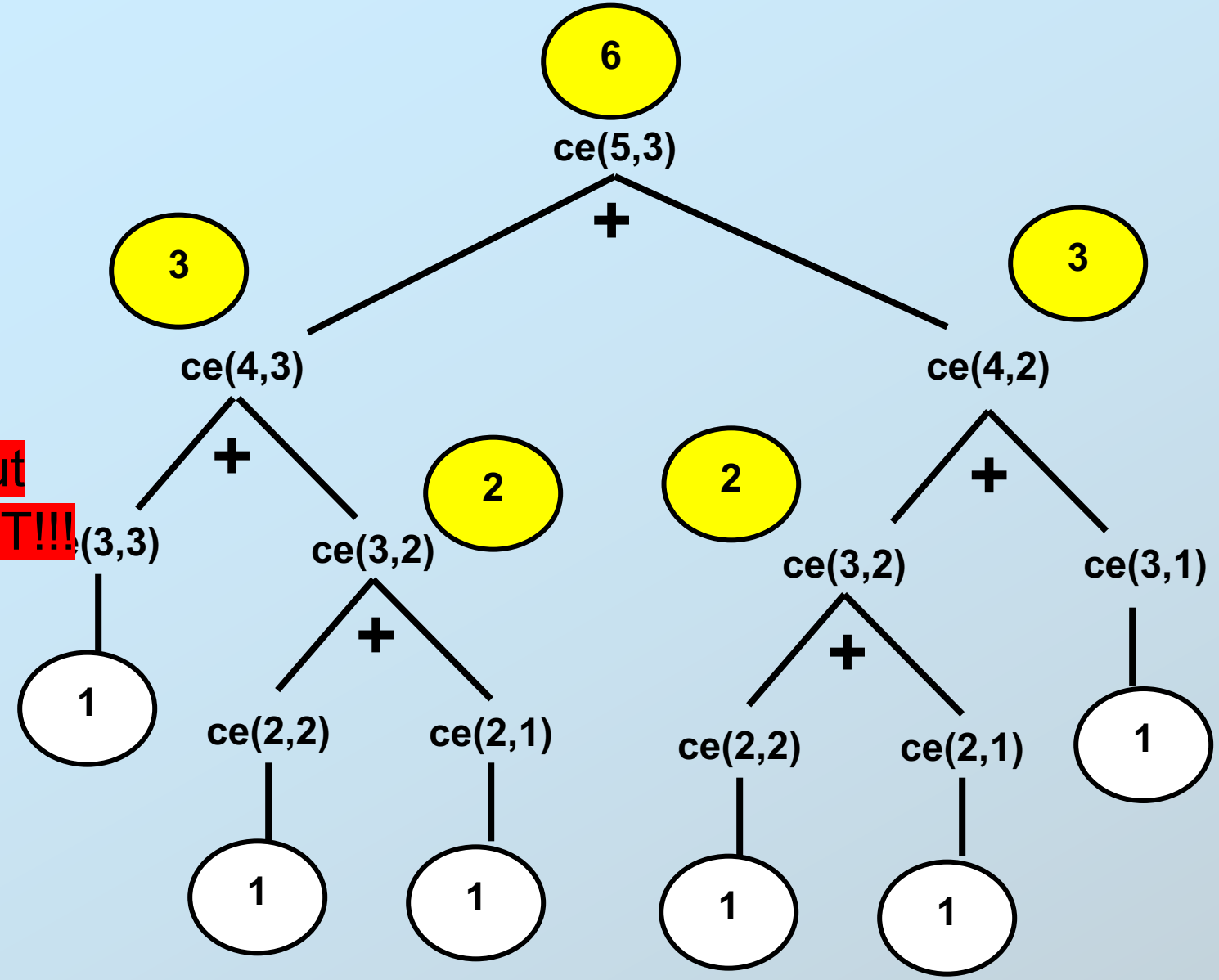
```
public class Recursive_Pascal {

  private static int compute_element (int row, int elem) {
      if (elem == 1) return 1;
      else if (elem == row) return 1;
      else return compute_element(row-1, elem) + compute_element(row-1, elem-1);
  }


 public static void Pascal (int N){
    for (int i = 1; i <= N; i++){
            System.out.println();
            for(int j=1;j<=i;++j)
                  System.out.printf("%10d", compute_element(i,j));
    }
 }


public static void main (String[] args){
       int N = Integer.parseInt(args[0]);
       Pascal(N);
       System.out.println();
  }
}
```

compute_element : ce



Easy to write but
NOT EFFICIENT!!!

```
public class Recursive_Pascal {

  private static int [][]memo; // αργότερα

  private static int compute_element (int row, int elem) {
      if (memo[row][elem]!=0) return memo[row][elem];
      if (elem == 1) return 1;
      else if (elem == row) return 1;
      else {
          int x =  compute_element(row-1, elem);
          int y= compute_element(row-1, elem-1);
          return (memo[row][elem]=x+y);
      }
  }

 public static void Pascal (int N)
    memo = new int[N+1][N+1];
    for (int i = 1; i <= N; i++){
            System.out.println();
            for(int j=1;j<=i;++j)
                  System.out.printf("%10d", compute_element(i,j));
   }
 }


public static void main (String[] args){
       int N = Integer.parseInt(args[0]);
       Pascal(N);
       System.out.println();
  }
}
```

MEMOIZATION
(Dynamic Programming)
Αποφυγή ίδιου υπολογισμού
Χρήση μνήμης

Σύνοψη

- Πώς να γράφετε απλά προγράμματα αναδρομής;
  - Βασική περίπτωση, βήμα αναδρομής
  - Ιχνηλατήστε την εκτέλεση ενός προγράμματος αναδρομής
- Γιατί να μάθετε αναδρομή;
  - Νέος τρόπος σκέψης
  - Ισχυρό προγραμματιστικό εργαλείο
- *Διαίρει και Βασίλευε:* Κομψή λύση σε πολλά σημαντικά προβλήματα
  - Και πολλά άλλα πλεονεκτήματα που ήδη αναφέραμε σε προηγούμενες ενότητες

Στο βιβλίο

- **Ενότητα 2.3**